

Programming Project 2: Set Operations

Required output/testability constraints

Size : Need a set equal to or greater than 10

Representation:

The bit string / boolean representation

Element name listing

Representative runs: In your Results section, include at least **2–3 representative test cases**. For Part 2, ensure both A and B include at least **two elements with multiplicity > 1**.

- Part 1
 - Given subsets of A and B of a set with N elements, use bit strings or an array of booleans to find
 - NOT(A)
 - A union B
 - A intersection B
 - A - B
 - Symmetric different of A B (A-B) union (B-A)
 - Bit strings can be created by boolean checks to see if element is in the set or not
 - Ex
 - Set :{A,B,C,D,E,F,G,H,I,J,K}
 - A {A,D,F,K} = (10010100001)
 - Yes a, no b, no c, yes d, ect)
 - B {A,B,D,E,F,G,K} = (11011110001)
 - Use binary operations
 - USE logical operators
 - ||, && !=
 - Your output should be clearly labeled so that someone reading the report can tell which operation produced which bit string / Boolean array.
 - Part 2
 - Sometimes the number of times an element occurs in an unordered collection matters. A multiset, m-set, or bag is an unordered collection of elements where an element can occur as a member more than once. Given multisets A and B from the same universal set, find:

- **Multiset union ($A \cup B$):** for each element, take the maximum count in A and B.
- **Multiset intersection ($A \cap B$):** for each element, take the minimum count in A and B.
- **Multiset difference ($A - B$):** for each element, subtract B's count from A's count, but not below 0.
- **Multiset sum ($A + B$):** for each element, add the two counts.
- FOR part 2, instead of using multiset arrays, use int arrays
- Create new methods for each multiset operations
 - Use Math functions `.max` and `.min` for union and intersection
 - Difference cannot go below 0 (it is binary)
 - Sum is easy enough